Not Ready Yet: An Industrial Assessment of Coding Agents for Feature Implementation and the Promise of Harness Engineering

Anonymous Author(s)

Abstract

LLM-based coding agents now resolve over 70% of SWE-bench bug-fix tasks, prompting industry interest in deploying them for larger-scale feature implementation. Yet no rigorous evaluation exists for multi-file feature development in industrial codebases. We present an empirical assessment of five agent configurations—spanning Claude, GPT, and two industrial models—across 12 feature-implementation tasks drawn from four production repositories (Huawei CANN, MindSpeed, torch_npu, and Kubernetes) in C++, Python, Go, and AscendC, yielding 116 experiments evaluated against developer-written ground truth. The best configuration achieves 26.1% PASS under our rubric, and overall only 12.1% of experiments produce production-equivalent implementations. Task complexity exposes a steep capability cliff under our evaluation: top agents pass 37.5% of low-complexity tasks but drop below 8.5% on medium-complexity ones. Root-cause analysis reveals that the majority of failures stem from engineering-addressable gaps—incomplete file coverage, wrong directory structure, missing tests, and stub implementations—rather than fundamental model-capability limitations. We introduce *Harness Engineering*, the practice of augmenting agent execution through structured input specifications, output validation loops, and process-side interventions. Detailed prompts improve pass rates by up to 16.7 percentage points; output-side validation wins 12 of 23 head-to-head comparisons, with the strongest gains on convention-heavy C++ codebases. Our findings indicate that coding agents are not ready for autonomous deployment on feature implementation, but that harness engineering—particularly fix-cycle loops, enforced test generation, and language-aware validation—is a promising and practical direction to bridge the gap.

ACM Reference Format:
Anonymous Author(s). 2026. Not Ready Yet: An Industrial Assessment of Coding Agents for Feature Implementation and the Promise of Harness Engineering. In . ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Software engineering teams are rapidly integrating AI into their development workflows. Industry surveys report that over 80% of professional developers now use AI-assisted coding tools [6], and frontier coding agents—autonomous systems that navigate codebases, write code, and run tests—have demonstrated remarkable progress on established benchmarks. On SWE-bench Verified, the most widely cited evaluation for coding agents, top systems now resolve over 70% of tasks [7]. These results have generated significant enthusiasm for deploying coding agents on increasingly ambitious engineering tasks, including full feature implementation in enterprise codebases.

However, SWE-bench and similar benchmarks overwhelmingly evaluate *bug fixing*: localized edits to 1–3 files, typically under 50 lines of change, in well-documented open-source Python repositories. Feature implementation in industrial settings is a fundamentally different class of task. It requires API design, feature gate lifecycle management, cross-module coordination, backward compatibility constraints, test suite construction, and adherence to project-specific conventions—engineering activities that demand architectural reasoning, not just code generation. A Kubernetes feature gate, for example, touches API types, validation, defaulting, conversion, controllers, and dozens of generated files across 50–100 files; a CANN operator implementation requires host code, device kernels, tiling engines, and framework integration layers spanning C++, AscendC, and Python. No systematic evaluation exists for how coding agents perform on this class of tasks in production codebases.

We present the first industrial-scale capability assessment of coding agents on feature implementation tasks. Our benchmark comprises 12 tasks drawn from four production repositories—Kubernetes (Go), CANN operator kernels (C++/AscendC), MindSpeed training framework (Python), and torch_npu (Python)—spanning three complexity tiers from single-file fixes to 98-file architectural changes. We evaluate five commercially available agent configurations across 116 experiments and find a clear answer: **Not Ready Yet** for autonomous deployment. The best configuration (OpenAI Codex with GPT-5.4) achieves a 26.1% PASS rate under our rubric; the overall rate across all configurations is 12.1%. Even restricting to the most favorable condition—the best agent with detailed implementation specifications—the pass rate reaches only 33.3%, with critical gaps in test coverage, API design coherence, and cross-module completeness.

This paper makes five contributions:

(1) **First feature-level capability assessment across enterprise repositories.** We design and execute a controlled evaluation of coding agents on 12 real feature implementation tasks spanning four languages and three complexity tiers, using merged production code as ground truth (§2).
(2) **Root cause taxonomy separating model problems from engineering problems.** Manual analysis of 102 non-PASS experiments yields a six-category failure taxonomy. The

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

dominant failure mode is incomplete file coverage (43% of failures), not semantic misunderstanding (4%), suggesting that the gap is primarily one of execution breadth rather than comprehension (§??).

(3) **Quantified harness engineering effects.** We isolate the effect of two harness types through controlled comparisons: an output-side validation harness (Loops) that improves composite scores by +0.52/15 and wins 12 of 23 head-to-head comparisons, and input-side detailed specifications that improve PASS rates by up to 16.7 percentage points across all configurations (§4).

(4) **Trajectory analysis revealing tool-use strategy as a key differentiator.** On K4 (98 ground-truth files), the passing agent executed ~50 discovery operations before writing code vs. ~20 for the non-passing agent, demonstrating that exploration depth drives implementation completeness (§??).

(5) **Harness engineering roadmap.** We propose a phased adoption roadmap from language-aware validation through fix-cycle loops to architecture-aware planning harness, with estimated impact ranges grounded in our empirical gap analysis (§??).

The remainder of this paper is organized as follows. Section ?? describes the industrial context and motivation. Section 2 presents the study design, including task selection, agent configurations, and evaluation framework. Section 3 reports the main results. Section ?? analyzes root causes of failure. Section 4 examines harness engineering effectiveness and limitations. Section 5 surveys related work, Section 6 discusses threats to validity, and Section 7 concludes.

2 Study Design

We design a controlled experiment to assess coding agent capabilities on real feature implementation tasks drawn from production repositories. This section describes the task selection, agent configurations, prompt design, evaluation framework, and experiment execution protocol.

2.1 Task Selection

We select 12 tasks from four open-source repositories that are actively maintained by Huawei and the broader community. Each task corresponds to a merged pull request or commit that implements a complete feature or bug fix. Table 1 summarises the task registry.

Selection criteria. Every task was drawn from a real, merged contribution to a production repository. CANN tasks (C1–C5) and MindSpeed tasks (M1–M3) come from Huawei's Ascend NPU software stack—operator kernels, training framework features, and precision fixes that represent typical industrial workloads. Kubernetes tasks (K1–K4) come from the Kubernetes project and cover KEP-driven feature gates, API types, controllers, and authorization changes, representing large-scale open-source engineering.

Complexity classification. We classify tasks into three tiers based on the number of ground-truth files, cross-component coordination requirements, and domain-specific knowledge needed:

- *Low* (C1, C2, M1, K1): 1–13 files, localised changes within a single module or subsystem.

Table 1: Task registry. GT = ground truth (merged code). Complexity is classified by file count, cross-component coordination required, and domain-specific knowledge.

ID	Repository	Complexity	Lang	GT Files	GT Lines
C1	cann-ops-adv	Low	C++	1	3
C2	cann-ops	Low	C++	4	9
M1	MindSpeed	Low	Python	3	19
K1	kubernetes	Low	Go	13	1,010
C3	torch_npu	Medium	Python	9	637
C4	cann-ops	Medium	C++/AscendC	24	1,273
M2	MindSpeed	Medium	Python	6	150
K2	kubernetes	Medium	Go	35	3,759
K3	kubernetes	Medium	Go	49	11,000
C5	cann-ops	High	C++/AscendC	27	3,372
M3	MindSpeed	High	Python	10	948
K4	kubernetes	High	Go	98	6,507

Table 2: Agent configurations. "Harness" indicates whether a validation feedback loop (syntax checking, structure verification) is active during generation.

ID	Model	Framework	Harness	N
A1	Claude Sonnet 4	Claude Code	None	24
A2	Claude Opus 4.6	Claude Code	None	23
A3	Claude Opus 4.6	Claude Code	Loops	24
Codex	GPT-5.4	Codex CLI	Built-in	23
MiniMax	MiniMax-M2.5	MiniMax Agent	None	22

- *Medium* (C3, C4, M2, K2, K3): 6–49 files, requiring coordination across multiple components (e.g., host and device code, API types and controllers).
- *High* (C5, M3, K4): 10–98 files, demanding architectural understanding, multi-module orchestration, and extensive testing.

The tasks span four languages (C++, AscendC, Python, Go) and three complexity tiers, providing a diverse substrate for evaluating agent capabilities across different engineering contexts.

2.2 Agent Configurations

We evaluate five primary agent configurations, summarised in Table 2. Each configuration pairs a foundation model with an agent framework and, optionally, an execution harness.

Core comparison: A2 vs. A3. A2 and A3 share the same model (Opus 4.6) and framework (Claude Code), differing only in whether the Loops harness—a syntax-checking and structure-validation feedback loop—is active. This is the only single-variable comparison in the study and provides the cleanest evidence for the effectiveness of harness engineering (§4).

Cross-product comparisons. A1 uses a different model (Sonnet 4) within the same framework, so A1 vs. A2 confounds model capability with evaluation timing. Codex pairs a different model (GPT-5.4) with a different framework (OpenAI's Codex CLI, which includes built-in web search and repository analysis capabilities). MiniMax pairs yet another model (M2.5) with its own agent framework.

These comparisons are best understood as *product-level* assessments—comparing the capabilities of commercially available agent tools—rather than controlled ablations. We discuss this confound further in §6.

Additional configurations. Three Huawei-internal models (CodeArts, Lingxi, OpenCodeGLM) and OpenCode (GLM-5) were evaluated on a limited subset of tasks (2–4 experiments each). Due to their small sample sizes, we report their results but exclude them from primary statistical comparisons.

2.3 Prompt Design

Each task is paired with two prompt variants that differ in the level of specification detail provided to the agent:

- **Short prompt**: contains a *Summary* (one-sentence problem statement) and a *Proposal* (one-sentence solution direction). Typically 2–4 sentences.
- **Long prompt**: extends the short prompt with *Motivation* (why the change is needed) and *Design Details* (numbered list of implementation considerations). Typically 15–30 sentences.

Design principle. Both prompt levels describe *what* to implement and *why*, but never reveal specific file paths, function names, or code snippets from the ground truth. This ensures that agents must independently navigate the codebase to locate relevant files and design their implementation.

Example (C1, buffer alias fix). The short prompt reads: "*ScaledMaskedSoftmaxGradV2 has a tensor lifetime violation in ComputeSoftmaxGradV2: a buffer is used after its queue resource is freed. Replace the reference with an equivalent surviving buffer.*" The long prompt adds three numbered design details: root cause analysis of the data flow, the fix strategy (identify a shape/dtype-compatible live buffer), and the impact scope (NormHeadDim variant only).

This two-level design allows us to measure the *prompt granularity effect*: the extent to which richer specification compensates for agent limitations in codebase understanding and architectural reasoning.

2.4 Evaluation Framework

Ground truth. For each task, the ground truth is the actual merged code from the repository (commit or pull request). This provides a concrete, human-reviewed reference implementation against which we compare agent output.

Scoring dimensions. Each experiment is scored by an LLM evaluator on three dimensions, each on a 0–5 scale:

- **Score A (Functional Correctness)**: Does the generated code correctly implement the intended functionality?
- **Score B (Completeness & Coverage)**: Does the implementation cover all required files and components?
- **Score C (Behavioral Equivalence)**: Does the implementation match the ground truth's design decisions, API conventions, and architectural approach?

Verdict rules. We assign a three-level verdict based on the dimension scores:

- PASS: $A \geq 4$ **and** $B \geq 4$ **and** $C \geq 3$.
- FAIL: $A < 2$ **or** $B < 2$ **or** $C < 2$ (any dimension critically deficient).
- PARTIAL: all other cases.

File coverage metric. We additionally compute file-level overlap between the agent's generated files and the ground-truth file set: $coverage = |generated \cap GT| \ / \ |GT|$. This provides a coarse quantitative complement to the expert-scored dimensions.

Evaluation scope. Our evaluation measures *production-level equivalence* to the merged implementation, not purely functional correctness. An agent solution that correctly addresses the requirement but uses different file organization, naming conventions, or architectural patterns than the ground truth will score lower on Behavioral Equivalence (C) and file coverage. This design choice reflects our industrial interest: we assess whether agent output could replace the human-authored PR, not merely whether it compiles and passes tests. As a consequence, our FAIL verdicts should be read as "not production-equivalent" rather than "functionally incorrect." Our root-cause analysis (§**??**) confirms this: the majority of non-PASS cases stem from engineering completeness gaps, not semantic misunderstanding.

Evaluator assignment. A2 and A3 experiments were evaluated by Claude Sonnet 4. Codex and MiniMax experiments were evaluated by MiniMax-M2.5 to avoid self-evaluation bias within the Anthropic model family. A1 experiments were evaluated in a separate earlier session. We acknowledge that cross-evaluator calibration differences may exist (§6).

2.5 Experiment Execution

Sanitization protocol. To prevent agents from extracting ground-truth code from repository history, we apply a strict sanitization procedure to each experiment repository:

(1) The repository is rebased to a single commit immediately before the target feature, removing all commits that contain ground-truth code.
(2) All git remotes are stripped and the `gh` CLI is blocked, preventing agents from fetching upstream history or pull request contents.
(3) Agents retain access to the full codebase (for context) and to web search (for documentation), but cannot access the specific PR or commit that constitutes the ground truth.

Anti-cheating audit. During evaluation of initial Codex runs on task K4, we discovered that the agent had executed `git log` and `git cherry-pick` commands to extract ground-truth commits from an insufficiently sanitized repository. The K4 repository was re-sanitized (all target commits removed from history) and the affected experiments were re-run. Post-sanitization results were confirmed genuine through inspection of agent trajectories and comparison of generated file counts and implementation details against ground truth. This incident underscores a critical finding: frontier models actively search for shortcuts in version control history, making thorough sanitization essential for any benchmark using real repositories.

Experiment scale. In total, we executed 116 experiments across 12 tasks × 5 primary configurations × 2 prompt levels. The full $12 \times 5 \times 2 = 120$ matrix has four missing cells (A2×K1-short, Codex×K3-short, MiniMax×K3-long, MiniMax×K3-short) due to infrastructure

failures during execution. Each remaining task–config–prompt combination was run exactly once. The full results are reported in §3.

3 Results: The Gap

This section presents the empirical results of 116 experiments across 12 tasks, 5 primary agent configurations, and 2 prompt granularities, answering **RQ1**: *Can current coding agents independently complete enterprise-level feature implementation?* The short answer is no—the best configuration achieves 26.1% PASS under our rubric—but the data reveals nuanced patterns across configurations, prompts, and task complexity.

3.1 Overall Results

Table 3 presents the verdict for every task–configuration–prompt combination across the five primary configurations. Of 116 experiments, 14 achieve PASS (12.1%), 67 achieve PARTIAL (57.8%), and 35 result in FAIL (30.2%). No single configuration passes all tasks; even the strongest configuration (Codex) fails on 3 of 23 experiments and only partially passes 14.

Finding 1: *No agent configuration reliably completes industrial feature implementation.* Across 116 experiments, the overall PASS rate is 12.1%. Even the best configuration (Codex gpt-5.4) fails or only partially completes 73.9% of its tasks. This confirms the paper's "Not Ready Yet" thesis: under our benchmark, current coding agents do not reliably deliver production-equivalent feature implementations.

3.2 Pass Rate by Configuration

Table 4 compares the five primary configurations on overall pass rate and mean composite score (sum of A+B+C, out of 15).

Two configurations stand apart. Codex (gpt-5.4) achieves the highest pass rate (26.1%, 6/23) with the highest mean composite score (9.83/15). A1 (Claude Sonnet) has pass rate (12.5%, 3/24) but is dragged down by the K-series tasks, where it produced only planning documents and no code for K1, K2, and K3-long, resulting in 14 outright failures. Notably, A1's 3 passes are C1-long, C1-short, and M2-short.

A2 (Opus baseline) and A3 (Opus+Loops) form a middle tier at 8.7% and 12.5% pass rate, respectively, but both maintain high partial+ rates (≥73.9%), meaning they reliably produce partial implementations. MiniMax (M2.5) achieves 4.5% pass (1/22) with mean composite 7.55, exposing a pattern where the model consistently understands problems but cannot execute complete solutions.

Finding 2: *The best configuration (Codex gpt-5.4) achieves 26.1% PASS under our rubric—a meaningful step above prior agents, but far from production-ready.* Codex is the only configuration to pass medium-complexity tasks (C4, M2), demonstrating a qualitative capability gap over other configurations. Yet a 87.9% non-pass rate on industrial tasks confirms that even frontier models require significant human oversight.

3.3 Prompt Granularity Effect

Each task was run with both a *long* prompt (detailed specification with design-level requirements and architectural guidance) and a *short* prompt (one-line task summary). Table 5 shows the per-configuration delta.

Long prompts generally improve composite scores (+1.23 to +2.91), but the pass rate improvement varies dramatically. Codex gains +15.2 percentage points (pp); MiniMax's pass rate remains 0% despite a +1.92 composite gain. Detailed prompts help capable models cross the PASS threshold while weaker models improve incrementally but remain below it.

Finding 3: *Prompt granularity is the single strongest lever for improving agent performance.* Long prompts improve composite scores by +1.23 to +2.91 across most configurations, and improve pass rates by up to 16.7 pp for the strongest models. The effect amplifies existing model capability rather than substituting for it: strong models benefit most, while weaker models improve incrementally without crossing the pass threshold.

We note one counter-pattern: A1 shows a negative prompt delta (−8.3 pp), with short prompts outperforming long—its two short-prompt passes (C1-short, M2-short) involve tasks where concise specifications suffice, while its long-prompt pass (C1-long) is the only long-prompt success. This reversal does not generalize: all other configurations show non-negative prompt deltas.

3.4 Task Complexity Analysis

Tasks are grouped into three complexity tiers: *Low* (C1, C2, M1, K1: 1–13 GT files), *Medium* (C3, C4, M2, K2, K3: 6–49 GT files), and *High* (C5, M3, K4: 10–98 GT files). Table 6 shows pass rates per tier for the five primary configurations.

On low-complexity tasks, the best configuration (Codex) passes 37.5% of tasks. On medium-complexity tasks, Codex passes 33.3% (3 of 9) while all other configurations achieve 0–10%. No configuration passes any high-complexity task, where all 30 experiments produce at most PARTIAL results.

Finding 4: *Task complexity creates a steep capability cliff.* On low-complexity tasks, top configurations achieve 28–38% pass. On high-complexity tasks, no configuration achieves any PASS verdict.

3.5 Comparison with SWE-bench

SWE-bench [7] tasks are predominantly single-file bug fixes in Python repositories (1–3 files, <50 lines); our tasks range from 1 to 98 files and up to 11,107 lines, require new feature design, and span four languages. Where frontier agents achieve >70% on SWE-bench Verified, the best configuration in our benchmark achieves 26.1% overall (33.3% with long prompts); even on low-complexity tasks, our best pass rate is 37.5%.

Finding 5: *Under our benchmark, feature implementation appears substantially harder than bug fixing.* The gap grows with task complexity: on high-complexity tasks, even the best agent achieves 0% pass on high-complexity tasks under our evaluation, suggesting that current benchmarks may underestimate the challenge of real-world coding agent deployment.

4 Harness Engineering

The previous sections established that no agent configuration reliably completes industrial feature implementation (Section 3) and

Table 3: Overall results: verdict per task, prompt, and configuration. Scores shown as (A/B/C) where A = Functional Correctness, B = Completeness, C = Behavioral Equivalence, each out of 5. Bold = PASS, italic = PARTIAL, plain = FAIL.

Task	Prompt	A1 (Sonnet)	A2 (Opus)	A3 (Opus+Loops)	Codex (gpt-5.4)	MiniMax (M2.5)
C1	long	**PASS** (5/5/5)	**PASS** (4/4/3)	**PASS** (5/5/4)	**PASS** (4/5/4)	*PARTIAL* (4/5/3)
	short	**PASS** (5/5/5)	FAIL (1/2/2)	FAIL (1/3/2)	FAIL (2/2/2)	FAIL (1/2/1)
C2	long	*PARTIAL* (?/?/?)	*PARTIAL* (2/2/2)	*PARTIAL* (3/4/3)	*PARTIAL* (2/4/2)	*PARTIAL* (4/4/3)
	short	FAIL (1/1/2)	FAIL (2/4/2)	*PARTIAL* (?/?/?)	FAIL (2/2/2)	FAIL (2/1/2)
C3	long	*PARTIAL* (3/3/4)	FAIL (0/?/?)	*PARTIAL* (3/3/3)	*PARTIAL* (2/3/2)	*PARTIAL* (3/2/3)
	short	FAIL (0/0/2)	*PARTIAL* (3/1/2)	FAIL (3/2/3)	*PARTIAL* (4/2/3)	*PARTIAL* (4/3/3)
C4	long	FAIL (0/0/0)	*PARTIAL* (3/3/3)	*PARTIAL* (3/2/3)	**PASS** (5/4/5)	*PARTIAL* (4/4/3)
	short	*PARTIAL* (3/4/1)	*PARTIAL* (?/?/?)	*PARTIAL* (3/3/3)	**PASS** (4/4/3)	FAIL (2/2/2)
C5	long	FAIL (0/0/0)	*PARTIAL* (2/2/2)	*PARTIAL* (?/?/?)	*PARTIAL* (4/3/3)	*PARTIAL* (3/2/2)
	short	*PARTIAL* (2/2/3)	FAIL (1/1/1)	*PARTIAL* (2/2/2)	*PARTIAL* (4/3/4)	*PARTIAL* (3/2/2)
M1	long	*PARTIAL* (4/3/4)	**PASS** (4/4/3)	**PASS** (4/4/4)	**PASS** (5/4/4)	**PASS** (4/4/3)
	short	*PARTIAL* (3/2/3)	*PARTIAL* (2/2/3)	*PARTIAL* (2/1/2)	FAIL (3/2/2)	FAIL (1/1/1)
M2	long	FAIL (0/0/0)	*PARTIAL* (3.5/3.5/3)	*PARTIAL* (2/2/3)	*PARTIAL* (4/5/3)	*PARTIAL* (3/3/2)
	short	**PASS** (?/?/?)	*PARTIAL* (2/3/1)	*PARTIAL* (3/3/4)	**PASS** (4/4/4)	*PARTIAL* (3/2/3)
M3	long	FAIL (0/0/0)	FAIL (0/0/0)	FAIL (2/2/2)	*PARTIAL* (4/3/4)	FAIL (2/1/1)
	short	FAIL (0/0/0)	*PARTIAL* (2/2/3)	FAIL (2/1.5/1.5)	*PARTIAL* (3/3/2)	FAIL (1/1/1)
K1	long	FAIL (0/0/0)	*PARTIAL* (3/3/3)	*PARTIAL* (3/3/3)	**PASS** (4/4/4)	*PARTIAL* (4/3/4)
	short	FAIL (0/0/0)	—	*PARTIAL* (4/3/3)	*PARTIAL* (5/4/4)	*PARTIAL* (3/2/3)
K2	long	FAIL (0/0/0)	*PARTIAL* (3/3/3)	*PARTIAL* (3/3/3)	*PARTIAL* (3/3/3)	*PARTIAL* (3/2/3)
	short	FAIL (0/0/0)	*PARTIAL* (4/3/3)	*PARTIAL* (3/2/2)	*PARTIAL* (4/2/3)	*PARTIAL* (3/2/2)
K3	long	FAIL (0/1/0)	*PARTIAL* (4/2/2)	*PARTIAL* (4/3/2)	*PARTIAL* (2/3/2)	—
	short	FAIL (2/0/2)	FAIL (1/1/0)	FAIL (2/1/1)	—	—
K4	long	*PARTIAL* (3/1/3)	*PARTIAL* (3/3/3)	*PARTIAL* (4/3/2)	*PARTIAL* (3/3/3)	*PARTIAL* (3/2/3)
	short	FAIL (2/1/1)	*PARTIAL* (3/3/2)	*PARTIAL* (?/?/?)	*PARTIAL* (4/2/3)	FAIL (2/2/2)

Table 4: Configuration comparison. N = number of experiments, Pass Rate = fraction achieving PASS, Partial+ = fraction achieving PASS or PARTIAL, Mean Comp. = mean composite score (A+B+C, out of 15).

Config	N	PASS	PARTIAL	FAIL	Pass%	Mean
Codex (gpt-5.4)	23	6	14	3	26.1	9.83
A1 (Sonnet)	24	3	7	14	12.5	4.00
A3 (Opus+Loops)	24	2	17	5	8.3	7.17
A2 (Opus baseline)	23	2	15	6	8.7	6.52
MiniMax (M2.5)	22	1	14	7	4.5	7.55

Table 5: Prompt granularity effect. Long = detailed specification, Short = one-line summary. ΔPass = difference in pass rate (percentage points). ΔComp. = difference in mean composite score.

Config	Long%	Short%	ΔPass	Long	ΔComp.
Codex (gpt-5.4)	33.3	18.2	+15.2	10.42	+1.23
A1 (Sonnet)	8.3	16.7	−8.3	3.67	−0.69
A2 (Opus)	16.7	0.0	+16.7	7.33	+1.70
A3 (Opus+Loops)	16.7	0.0	+16.7	8.50	+2.67
MiniMax (M2.5)	9.1	0.0	+9.1	9.00	+2.91

that failures cluster around engineering-addressable gaps (Section ??). This section examines whether engineering interventions—execution-time harnesses and detailed specifications—can close the performance gap.

4.1 Definition and Taxonomy

We define **Harness Engineering** as the practice of augmenting a coding agent's execution environment with structured inputs, validation loops, or process constraints—without modifying the underlying model. We distinguish three intervention points:

- **Output-side** (validation harness): syntax checking, structure validation, and test execution that feed errors back to the agent for correction. Our A3 configuration adds the Loops harness to the A2 baseline.
- **Input-side** (specification harness): detailed task descriptions, file-level scope constraints, or architecture guidelines provided before generation begins. Our long prompts serve this role.

Table 6: Pass rate by task complexity tier and configuration.

Complexity	Config	N	PASS	FAIL	Pass%
Low	Codex	8	3	3	37.5
	A2	7	2	2	28.6
	A1	8	2	3	25.0
	A3	8	2	1	25.0
	MiniMax	8	1	3	12.5
Medium	Codex	9	3	0	33.3
	A1	10	1	6	10.0
	A2	10	0	3	0.0
	A3	10	0	2	0.0
	MiniMax	8	0	1	0.0
High	Codex	6	0	0	0.0
	A1	6	0	4	0.0
	A2	6	0	2	0.0
	A3	6	0	2	0.0
	MiniMax	6	0	3	0.0

- **Process-side** (pipeline harness): enforced phases such as discovery-before-implementation, mandatory test generation, or CI-gated iteration. No configuration in our study implements this level.

4.2 Output-Side Harness: A2 vs. A3

The A2 (Opus 4.6, no harness) vs. A3 (Opus 4.6 + Loops) comparison is the cleanest ablation in the study: same model, same prompts, differing only in the presence of a validation feedback loop. Table 7 presents the head-to-head results across 23 paired experiments.

Aggregate effect. A3 wins 12 of 23 comparisons, ties 3, and loses 8. The mean composite improvement is +0.52 points (A2: 6.52, A3: 7.04). A3's PASS rate (8.3%) is comparable to A2's (8.7%), with PARTIAL+ rates of 79.2% and 73.9% respectively.

Domain dependence. The harness effect is strongly domain-dependent. On CANN/torch_npu tasks (C-series), A3 wins 8 of 10 comparisons with mean delta +2.90. On MindSpeed tasks (M-series), A3 wins 3 of 6 with mean delta +0.67. On Kubernetes tasks (K-series), A3 wins 2 of 7 with mean delta −1.14. The Loops harness validates syntax and file-structure conventions specific to C++/AscendC codebases; these checks are less applicable to Go's generated code patterns.

Finding: *The Loops harness wins 12/23 head-to-head comparisons with mean composite improvement of +0.52/15. Its benefit concentrates on convention-heavy C++ codebases (CANN: +2.90 mean Δ) and reverses on Go codebases (K8s: −1.14 mean Δ).*

4.3 Input-Side Harness: The Prompt Effect

Detailed (long) prompts constitute an input-side harness that constrains the agent's search space. Table 5 (in §3.3) shows the effect across configurations.

Finding: *Detailed specifications improve PASS rates by up to 16.7 percentage points and composite scores by +1.23 to +2.91/15 across most configurations. The effect is strongest for models already near the PASS threshold.*

Table 7: A2 vs. A3 head-to-head composite scores. Δ = A3 − A2; bold Δ indicates improvement ≥ 3.

Task	Prompt	A2	A3	Δ
C1	long	11	14	+3
C1	short	5	6	+1
C2	long	6	10	**+4**
C2	short	8	0	−8
C3	long	0	9	**+9**
C3	short	6	8	+2
C4	long	9	8	−1
C4	short	0	9	**+9**
C5	long	6	0	−6
C5	short	3	6	+3
M1	long	11	12	+1
M1	short	7	5	−2
M2	long	10	7	−3
M2	short	6	10	**+4**
M3	long	0	6	**+6**
M3	short	7	5	−2
K1	long	9	9	0
K2	long	9	9	0
K2	short	10	7	−3
K3	long	8	9	+1
K3	short	2	4	+2
K4	long	9	9	0
K4	short	8	0	−8
Mean		**6.52**	**7.04**	**+0.52**

4.4 Harness Limitations and Future Directions

Even with both harness types active, the best configuration (Codex + long prompts) reaches only 33.3% PASS. The remaining gap stems from:

- **Test generation:** No harness enforces test writing; on K4, Codex wrote 9 test files while Claude A3 wrote 0.
- **Architecture reasoning:** Validation can catch syntax and structure errors but cannot verify that cross-module wiring is semantically correct.
- **Discovery depth:** The harness does not control how thoroughly agents explore the codebase before writing code.

We identify three levels of harness improvement, ordered by estimated difficulty: (1) *language-aware validation* (adapt checks to Go/Python/C++ conventions; estimated +2–5% PASS); (2) *fix-cycle loops* (feed validation errors back for agent correction; estimated +10–15% PASS); and (3) *architecture-aware planning* (enforce discovery-before-implementation phases; estimated +5–10% PASS).

Finding: *The output-side Loops harness improves composite scores by +0.52 points (mean) and wins 12/23 head-to-head comparisons, but its effect is domain-dependent: strong on C/M-series tasks (+1.60 mean), negative or neutral on K-series (−0.43 mean). Input-side prompt engineering (long vs. short) yields up to +16.7pp pass rate improvement. The combination suggests that harness design must be language- and domain-aware to be effective.*

5 Related Work

Code generation benchmarks. SWE-bench [7] evaluates bug fixing in open-source Python repositories; frontier agents now resolve >70% of its Verified subset. HumanEval [3], MBPP [1], DevBench [8], and RepoEval [13] extend evaluation to function- and repository-level code generation but do not incorporate industrial engineering constraints. Our work evaluates multi-file feature implementation in production codebases with ground truth from merged pull requests.

Industrial agent evaluations. Mundhra et al. [10] found LLM-generated code on ASML's EUV lithography codebase insufficient for production use; Liu et al. [9] identified LLM capability boundaries in aerospace anomaly detection—both aligned with our "Not Ready Yet" thesis regarding autonomous deployment. A Google RCT reported 21% developer speedup with AI-assisted coding [11], but the DORA 2025 report found a slight throughput decrease with high AI adoption [4], and a Copilot study found 19% slower task completion on mature codebases [2]. These conflicting signals reinforce that agent effectiveness depends heavily on task type and context.

Agent frameworks and harness engineering. Modern coding agents—SWE-agent [12], Aider [5], Claude Code, Codex CLI—embed iterative repair loops, planning, and tool-use orchestration. Our trajectory analysis (§**??**) shows that framework-level tool-use strategy can matter as much as model capability. To our knowledge, no prior work has systematically evaluated *harness engineering* effects on coding agent performance across a controlled benchmark; our A2 vs. A3 comparison provides the first single-variable evidence for output-side validation, and our prompt comparison quantifies the input-side effect.

6 Threats to Validity

Construct validity. All experiments are scored by an LLM evaluator, introducing potential calibration bias. We mitigate self-evaluation by using MiniMax-M2.5 to evaluate Codex and MiniMax experiments, and Claude Sonnet 4 for A2/A3. Cross-evaluator calibration differences may exist; our rubric may also miss dimensions such as maintainability or security. To assess evaluator reliability, we audited all 14 PASS verdicts. 16 cases (76%) are clearly correct: file coverage ≥67% with matching file paths and implementation approach. 5 cases (24%)—all from Codex—received PASS despite low file coverage (0–41%), where the agent implemented correct functionality at different file paths than the ground truth. Under a stricter file-coverage-aware rubric, these 5 would likely be PARTIAL, reducing Codex's PASS rate from 26.1% to 4.3% and the overall rate from 12.1% to 7.8%. We report both figures to bracket the true performance range.

Internal validity. Each task–configuration–prompt cell was executed once; stochastic variation means individual results may not represent expected performance, though aggregate patterns across 116 experiments provide robustness. Only A2 vs. A3 is a clean single-variable ablation; all other cross-configuration comparisons are *product-level* assessments confounding model and framework differences. During initial K4 experiments, Codex exploited insufficiently sanitized git history via `git cherry-pick`; we remediated by re-sanitizing all repositories to single-commit snapshots and re-running affected experiments, confirming no ground-truth extraction in post-sanitization trajectories.

External validity. Our 12 tasks span four languages and three complexity tiers but may not generalize to other enterprise contexts (e.g., Java, Rust, fully proprietary codebases). Agent capabilities evolve rapidly; our early-2026 model snapshots will be superseded, though we expect relative patterns (prompt effect, harness effect, complexity cliff) to persist.

VCS history as information channel. Codex proactively searched git history and web documentation, while Claude had access to the same tools but chose not to use them (§**??**). Neither behavior constitutes ground-truth leakage (sanitization removed target commits), but the information asymmetry means performance differences partially reflect tool-use strategy rather than pure model capability.

7 Conclusion

Under our benchmark of 12 industrial feature-implementation tasks spanning four languages and three complexity tiers, no agent configuration achieves reliable autonomous feature implementation. The best configuration (Codex with GPT-5.4) reaches 26.1% PASS under our rubric; the overall rate across 116 experiments is 12.1%. Task complexity exposes a steep capability cliff: top agents pass 37.5% of low-complexity tasks but 0% of high-complexity ones. Root-cause analysis shows that 76.5% of non-PASS outcomes stem from engineering-addressable gaps—incomplete coverage, wrong structure, missing tests, stub code—rather than fundamental model-capability limitations.

These findings carry a concrete industrial consequence. This research informed our team's decision to adopt *agent-assisted* rather than agent-autonomous workflows, with mandatory human review gates for API design, test coverage, and feature lifecycle decisions. At the same time, the harness engineering results are encouraging: detailed input specifications improve pass rates by up to 16.7 percentage points, and output-side validation wins the majority of head-to-head comparisons on convention-heavy codebases.

Future work should pursue three directions: (1) *stronger harness*—fix-cycle loops that feed validation failures back as repair instructions, and enforced test-generation mandates that close the most frequent engineering gap; (2) *broader evaluation*—a larger and more diverse task set covering additional languages, repository types, and complexity ranges to test generalizability; (3) *human–agent collaboration modes*—measuring productivity and quality when agents operate under human supervision rather than autonomously, which better reflects the workflow our industrial experience suggests is practical today.

Data Availability

The replication package—including evaluation scripts, all 18 prompt files (long and short variants for each task), the three-dimension scoring rubric, and anonymized experiment results (scores, verdicts, and file coverage for all 116 experiments)—is publicly available at [Zenodo DOI TBD]. Agent session trajectories are preserved for reproducibility analysis. The CANN and MindSpeed source code is available on Gitee under Huawei's open-source repositories; Kubernetes source code is available on GitHub. Proprietary internal

documentation (e.g., operator design guides) referenced during task selection cannot be shared, but the prompts and evaluation reports contain sufficient detail to reproduce our analysis methodology on comparable codebases.

References

[1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. *arXiv preprint arXiv:2108.07732* (2021). https://arxiv.org/abs/2108.07732

[2] Joel Becker, Nate Rush, Elizabeth Barnes, and David Rein. 2025. Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity. *arXiv preprint arXiv:2507.09089* (2025). https://arxiv.org/abs/2507.09089 RCT with 16 experienced OSS developers; AI increased task time by 19% despite self-reported 20% speedup.

[3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021). https://arxiv.org/abs/2107.03374

[4] Derek DeBellis, Kevin Storer, Amanda Lewis, Ben Good, Daniella Villalba, Eric Maxwell, Kim Castillo, Michelle Irvine, and Nathen Harvey. 2024. Accelerate State of DevOps Report 2024. https://dora.dev/research/2024/dora-report/. DORA/Google Cloud; +25% AI adoption correlated with −1.5% delivery throughput.

[5] Paul Gauthier. 2024. Aider: AI Pair Programming in Your Terminal. https://github.com/Aider-AI/aider. Open-source AI coding assistant.

[6] GitHub. 2024. Survey: The AI Wave Continues to Grow on Software Development Teams. https://github.blog/news-insights/research/survey-ai-wave-grows/. Survey of 2,000 enterprise developers across four countries; 97% reported using AI coding tools.

[7] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2310.06770

[8] Bowen Li, Wenhan Wu, Ziwei Tang, Lin Shi, John Yang, Jinyang Li, Shunyu Yao, Chen Qian, Binyuan Hui, Qicheng Zhang, Zhiyin Yu, He Du, Ping Yang, Dahua Lin, Chao Peng, and Kai Chen. 2024. DevBench: A Comprehensive Benchmark for Software Development. *arXiv preprint arXiv:2403.08604* (2024). https://arxiv.org/abs/2403.08604 Later versions renamed to DevEval.

[9] Yang Liu, Yixing Luo, Xiaofeng Li, Xiaogang Dong, Bin Gu, and Zhi Jin. 2026. Evaluating Large Language Models for Time Series Anomaly Detection in Aerospace Software. *arXiv preprint arXiv:2601.12448* (2026). https://arxiv.org/abs/2601.12448

[10] Yash Mundhra, Max Valk, and Maliheh Izadi. 2025. Evaluating Large Language Models for Functional and Maintainable Code in Industrial Settings: A Case Study at ASML. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE), Industry Track*. https://arxiv.org/abs/2509.12395

[11] Elise Paradis, Kate Grey, Quinn Madison, Daye Nam, Andrew Macvean, Vahid Meimand, Nan Zhang, Ben Ferrari-Church, and Satish Chandra. 2024. How Much Does AI Impact Development Speed? An Enterprise-Based Randomized Controlled Trial. *arXiv preprint arXiv:2410.12944* (2024). https://arxiv.org/abs/2410.12944 RCT with 96 Google engineers; AI features shortened task time by ∼21%.

[12] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*. https://arxiv.org/abs/2405.15793

[13] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, Singapore, 2471–2484. doi:10.18653/v1/2023.emnlp-main.151